Planner GRPO 설계를 이해하기 위한 개념 설명

1. 이 프로젝트에서 Agent가 해결하려는 문제

현재 시스템은 대략 다음과 같은 검색 파이프라인을 가진다.

```
Transaction
    ↓
Router
    ↓
Retriever
    ↓
최근접 정상 데이터
    ↓
Evaluator
    ↓
검색 결과에 부족한 부분이 있는지 판단
    ↓
Planner
    ↓
필요하다면 다른 관점으로 재검색
```

중요한 점은 **Agent의 목표가 anomaly detector 자체를 새롭게 만드는 것이 아니라는 것**이다.

AnoLLM embedding과 HNSW라는 강력한 dense retrieval system이 이미 존재하고 있으며, anomaly score인 `nd`도 존재한다. Agent가 해결하려는 문제는 다음과 같이 정의한다.

> **"현재 검색 결과만으로 충분한가? 충분하지 않다면 어떤 검색 관점으로 추가 evidence를 얻는 것이 가장 유용한가?"**

따라서 anomaly detection은 application-level outcome이고, 핵심 연구 대상은 **retrieval critique → retrieval planning → additional retrieval**이라는 agentic retrieval loop이다.

---

2. Evaluator와 Planner의 역할을 분리해야 하는 이유

기존 Evaluator는 검색 결과를 보고 `cluster_mismatch` 또는 `neighbor_insufficient`와 같은 상태를 판단한다.

하지만 자세히 보면 이 판단의 핵심 근거는 대부분 deterministic하다.

예를 들어 cluster persona가 다음과 같다고 하자.

```
numeric:
purchase_value z-score = +2.1
```

```
→ |z| > zτ=1.5 → mismatch

categorical:
source = Ads
IP_country = US
browser = Chrome
```

테스트 transaction이:

```
purchase_value = z=2.1
source = Ads
IP_country = US
browser = Safari
```

라면,

```
deviant = [purchase_value, browser]
agrees  = [source, IP_country]
```

라는 사실 자체는 LLM이 발견해야 하는 것이 아니다.

따라서 Evaluator가 해야 할 일은:

```
deterministic evidence
       ↓
structured interpretation
       ↓
"cluster_mismatch"
"deviant=[purchase_value,browser]"
"agrees=[source,IP_country]"
```

정도로 제한한다.

즉 Evaluator는 **tool을 선택하지 않는다.**

---

3. 왜 Evaluator에는 GRPO가 적합하지 않았는가

기존 실험에서는 teacher가 만든 정답 문장을 이용하여 Evaluator를 SFT하고, 이후 짧은 출력이나 표현 다양성 같은 reward를 추가하여 GRPO를 적용했다.

그런데 이 문제는 본질적으로:

```
deterministic numerical evidence
       ↓
정해진 classification
       ↓
정해진 textual explanation
```

에 가깝다.

예를 들어 실제 계산 결과가:

```
cluster mismatch
reason = zτ violation
```

인데 모델이:

```
neighbor insufficient
reason = high spread
```

라고 출력하면 틀린 것이다.

따라서 이 문제에서는 **정답 데이터가 존재하고 SFT가 mapping을 직접 학습할 수 있다.**

반면 `short token`, `표현 다양성` 같은 GRPO reward는 실제 retrieval utility와 직접 관계가 없다.

그래서 Evaluator에서 GRPO를 계속 시도하기보다는:

> **Evaluator = 정확한 evidence interpreter → SFT**

로 고정한다.

---

4. Planner가 새롭게 해결하는 문제

Planner에서는 상황이 달라진다.

Evaluator가 다음을 알려줬다고 하자.

```
wm_type = cluster_mismatch
deviant = [browser, purchase_value]
agrees = [source, IP_country]
nd = 0.72
```

이 정보만으로 **정답 action이 결정되는 것은 아니다.**

Planner는 다음 중 하나를 선택할 수 있다.

```
{"action":"stop"}
```

```
{"action":"global_knn"}
```

```
{"action":"contextual_knn",
 "features":["source"]}
```

```
{"action":"contextual_knn",
 "features":["source","IP_country"]}
```

어떤 것이 실제로 가장 좋은지는 검색을 실행해봐야 알 수 있다.

예를 들어:

```
stop
→ nd = 0.72

global_knn
→ nd = 0.91

contextual[source]
→ nd = 0.84

contextual[source, IP_country]
→ nd = 0.96
```

라면 마지막 action이 가장 좋은 선택이다.

그러나 다른 transaction에서는 global이 가장 좋을 수도 있다.

이것이 **Evaluator와 Planner의 핵심적인 차이**다.

```
Evaluator:
state → interpretation

Planner:
state → action
            ↓
          environment
```

```
        ↓
    outcome
```

Planner의 action은 실행 결과를 보기 전에는 정답을 완전히 알 수 없다.

---

5. Contextual KNN은 무엇을 의미하는가

`contextual_knn`은 embedding 자체를 변경하는 것이 아니다.

기존 AnoLLM embedding을 그대로 사용하면서 **원본 tabular feature를 검색 조건으로 추가한다.**

예를 들어:

```
현재 transaction

source = Ads
IP_country = US
browser = Safari
```

이고 Evaluator가:

```
agree = [source, IP_country]
deviant = [browser]
```

라고 판단했다면 Planner가:

```
{
 "action":"contextual_knn",
 "features":["source","IP_country"]
}
```

를 선택할 수 있다.

그러면 검색은 개념적으로:

```
dense similarity
       +
source=Ads를 만족하는 정도
       +
IP_country=US를 만족하는 정도
```

를 함께 고려한다.

중요한 것은 이것이 **hard filtering과 반드시 동일하지 않다는 것**이다.

현재 설계에서는 soft `should` boost를 사용하여:

> embedding similarity는 유지하면서 원본 feature context가 맞는 후보를 우선시한다.

따라서 이것은 **dense retrieval + structured feature signal의 hybrid retrieval**로 이해하는 것이 가장 적절하다.

---

6. Global KNN과 Contextual KNN의 차이

두 검색은 서로 다른 retrieval view를 제공한다.

Global KNN

```
제약 없음
→ 전체 정상 memory bank에서 검색
```

질문은:

> "현재 cluster나 feature context를 무시하면 전역적으로 더 적합한 정상 이웃이 있는가?"

이다.

Contextual KNN

```
전체 또는 지정 scope에서
embedding similarity
+
선택된 원본 feature context
```

질문은:

> "현재 transaction과 embedding뿐 아니라 특정 원본 feature context까지 고려하면 더 적합한 이웃이 있는가?"

이다.

따라서 Planner가 선택하는 것은 단순히 "검색할까 말까"가 아니라:

> **어떤 retrieval view를 사용할 것인가**

이다.

---

7. 왜 feature set까지 Planner의 action인가

`contextual_knn`만 선택하게 만들면:

```
contextual → 항상 같은 query
```

가 되어 policy의 자유도가 너무 작다.

예를 들어:

```
contextual(source)
contextual(source, IP_country)
contextual(source, sex)
```

는 모두 서로 다른 검색 결과를 만들 수 있다.

따라서 Planner가:

```
contextual_knn
+
어떤 feature를 유지할 것인가
```

를 함께 결정하도록 한다.

다만 0.5B 모델에게 무제한 조합을 주면 너무 어렵기 때문에:

```
features ⊆ evaluator가 제공한
           agree ∪ persona-match
```

그리고:

```
|features| ≤ 2
```

정도로 action space를 제한한다.

이렇게 하면 수백 개의 조합 문제가 아니라 약 10개 수준의 선택 문제가 된다.

---

8. Oracle은 무엇인가

Planner 학습에서 중요한 개념이 `oracle`이다.

모든 가능한 action을 실제로 실행해보고:

```
stop                  → R=0.00
global                → R=0.08
contextual[source]    → R=0.03
contextual[source,IP] → R=0.12
```

라면:

```
oracle = contextual[source,IP]
```

라고 한다.

하지만 이것을 **정답 label로 사용한다는 의미는 아니다.**

Oracle은:

> "이 state에서 모든 action을 미리 실행했다면 무엇이 가장 좋았는가?"

를 측정하기 위한 **counterfactual reference**이다.

실제 serving에서는 모든 action을 실행하지 않는다.

그래서 Planner는:

```
현재 state
    ↓
어떤 action을 실행할지 예측
```

해야 한다.

---

9. 왜 GRPO가 필요한가

이 구조에서 GRPO의 핵심은:

> **정답 action을 외워서 출력하는 것**

이 아니다.

동일한 state에서 여러 action을 시도하고 그 결과를 비교하여:

```
어떤 action이 실제 utility를 높였는가?
```

를 policy가 학습하도록 하는 것이다.

예를 들어:

```
Group

stop                  R=0.00
global_knn            R=0.05
contextual[source]    R=0.08
contextual[source,IP] R=0.12
```

이면 contextual `[source, IP]`가 높은 advantage를 얻는다.

다른 transaction에서는:

```
stop                  R=0.00
global_knn            R=0.15
contextual[source]    R=0.04
contextual[source,IP] R=0.01
```

이면 global이 높은 advantage를 얻는다.

따라서 Planner는:

```
cluster_mismatch → 항상 contextual
```

같은 규칙을 학습하는 것이 아니라,

> **현재 state에서 어느 retrieval action의 marginal utility가 높은가**

를 학습한다.

---

10. Reward의 핵심

현재 설계의 가장 중요한 reward는 **decision improvement**다.

Anomaly detection 관점에서는:

```
normal:
nd → threshold보다 더 안전한 normal 방향
→ positive reward

anomaly:
```

```
nd → anomaly 방향
→ positive reward
```

로 정의할 수 있다.

이를 간단히:

```
margin(nd,y) = (2y-1)(nd-τ)
```

로 표현하고,

```
Δmargin =
margin(after) - margin(before)
```

를 primary utility로 사용할 수 있다.

여기에 검색 비용을 추가한다.

```
R ≈ Δmargin - λ·cost
```

이것이 Stage 1의 핵심이다.

이후 필요성을 확인하면서:

```
Stage 1
decision improvement

Stage 2
+ degradation penalty

Stage 3
+ evidence novelty

Stage 4
+ validity/support penalty
```

로 확장한다.

중요한 것은 **처음부터 reward를 많이 넣지 않는 것**이다.

Evaluator에서 이미 경험했듯이 reward가 task utility와 직접 연결되지 않으면 GRPO가 좋은 policy를 만드는 대신 기존 SFT 성능을 흔들 수 있다.

---

11. Search Agent 관점에서 이 구조가 의미하는 것

일반적인 AgentRAG를 단순화하면:

```
query
 ↓
retrieve
 ↓
evaluate retrieved evidence
 ↓
what is missing?
 ↓
rewrite / choose another retrieval strategy
 ↓
retrieve
 ↓
evaluate
 ↓
stop
```

이다.

현재 프로젝트는 이것을:

```
transaction
 ↓
router + retriever
 ↓
evaluator
 ↓
what evidence is insufficient?
 ↓
planner
 ↓
global / contextual retrieval
 ↓
decision improvement
 ↓
stop
```

으로 대응시킨다.

따라서 anomaly 자체가 핵심 연구 문제가 아니라:

> **검색 결과를 critique하고, 부족한 evidence를 보충하기 위한 retrieval strategy를 선택하고, 그 결과를 outcome으로 평가하는 Agentic Retrieval Environment**

로 보는 것이 프로젝트의 가장 중요한 framing이다.

---

12. 이 프로젝트에서 반드시 기억해야 할 구분

가장 중요한 세 가지를 구분해야 한다.

Evaluator

> "현재 검색 결과에 무엇이 부족한가?"

→ 가능한 한 deterministic evidence 기반
→ SFT

Planner

> "그 부족함을 어떤 검색 행동으로 보완할 것인가?"

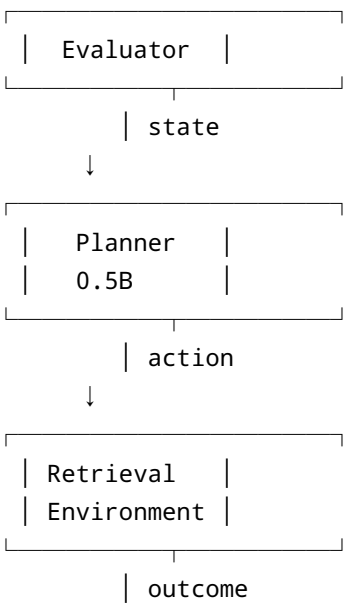
→ action 선택에 불확실성 존재
→ GRPO

Environment

> "그 action을 실행했더니 실제로 얼마나 좋아졌는가?"

→ nd 변화, cost, latency, support 등을 관찰

따라서 전체 구조는:



```
                ↓
            GRPO Reward
```

이다.

---

13. 최종적으로 실험해야 하는 핵심 질문

이 프로젝트에서 GRPO가 성공했는지를 단순히:

> "Planner가 evaluator의 판단을 잘 맞혔는가?"

로 평가하면 안 된다.

대신:

1. Planner가 `global/contextual/stop`을 상황에 따라 다르게 선택하는가?
2. 선택한 action이 실제 `Δmargin`을 증가시키는가?
3. 같은 utility를 더 낮은 cost로 얻는가?
4. global-only shortcut보다 좋은가?
5. oracle과 비교했을 때 regret가 감소하는가?

를 봐야 한다.

그리고 만약:

```
0.5B SFT-on-oracle ≈ GRPO
```

라면 그것도 중요한 결과다.

그 경우:

> "이 planner task는 RL보다 supervised policy learning으로 충분했다."

라고 결론 내리는 것이 맞다.

반대로 GRPO가 SFT보다 낮은 regret를 보인다면:

> **검색 결과를 보고 retrieval action을 동적으로 선택하는 데 RL이 실제로 추가적인 이점을 제공했다**

는 훨씬 강한 결과가 된다.